

Certified Normal Forms and Complete Relations for Circuit Presentations in Agda

ANONYMOUS AUTHOR(S)

Completeness proofs for circuit rewrite systems often rely on long calculations with generators, relations, normal forms, and case analyses over families of circuits. These calculations are exactly the sort of argument that is easy to get wrong on paper: a single missing commutation case or a mistaken field identity can invalidate the claimed presentation. We describe an Agda framework for formalizing circuit presentations as words modulo congruence relations, proving normal-form theorems for those presentations, and deriving complete rewrite systems by verified normalization.

The central case study is a parameterized formalization of the symplectic group $\text{Sp}(2n, \mathbb{Z}_p)$, where n is arbitrary and p is an odd prime. We define Pauli vectors, the symplectic form, the action of the generators S , H , and CZ , an inductive family of normal forms, implementability of every symplectic linear map by a normal form, and injectivity of normal forms. We then formalize the proof pattern that reduces arbitrary words to normal form by pushing generators through normal boxes. This yields a certified route from relations to completeness: two words are equal precisely when their normal forms coincide.

Beyond the symplectic development, the framework supports semidirect products, group extensions, coset normal forms, and amalgamated products. We use these constructions to derive Clifford-mod-scalar relations from symplectic and Pauli relations, simplify the Clifford presentation, explain how scalar phases can be added as a general group extension, and verify several independent amalgamation examples. The result is both a mechanized proof of a substantial qudit Clifford presentation and a reusable library for certified algebraic reasoning about circuit calculi.

Additional Key Words and Phrases: Agda, quantum circuits, normal forms, presentations, Clifford group, symplectic group

1 Introduction

Equational theories for circuits are useful only when their rule sets are complete: if two circuits denote the same operation, the equations should be able to transform one circuit into the other. Completeness is usually proved by choosing a normal form, showing that every circuit reduces to one, and proving that equal denotations have equal normal forms. This proof strategy is conceptually clean, but in quantum and reversible circuit calculi the actual proof is computationally dense. Even for well-structured fragments, the proof requires many parameterized relations, many interactions between gates on different wires, and many identities in finite rings or fields.

This paper describes a proof-assistant approach to these completeness proofs. Instead of performing the relation reductions by hand, we formalize presentations, words, normal forms, and proof-generating normalization in Agda [31]. The immediate payoff is reliability: the proof assistant checks every algebraic step used to derive a box relation or simplify a presentation. The broader payoff is reuse: once presentations and normal forms are expressed abstractly, the same proof patterns apply to semidirect products, coset normal forms, group extensions, and amalgamated products.

The main mathematical development is a mechanization of normal forms and complete relations for $\text{Sp}(2n, \mathbb{Z}_p)$, uniformly in the number of qudits n and the odd prime dimension p . The generators are the symplectic images of the qudit Clifford gates S , H , and CZ . The normal forms are built from inductively defined boxes. Each box has a semantic action on Pauli vectors, and each step of the normal-form proof is checked against the symplectic form. The formalization proves that every symplectic linear transformation preserving the form is implemented by a normal form, and that normal forms are unique.

The second focus is the path from the symplectic presentation to Clifford presentations. The Clifford group modulo scalars decomposes into a Pauli layer and a symplectic layer. We express this as a semidirect product presentation: the Pauli presentation forms the normal subgroup, the symplectic presentation acts by conjugation, and the combined relations specify how symplectic generators move past Pauli generators. We then simplify the resulting Clifford-mod-scalar relations by pushing Pauli components to canonical positions and proving an identity isomorphism between the original and simplified presentations. Finally, scalar phases are added using a more general group-extension construction: quotient relators may lift with scalar corrections, unlike the split semidirect-product case.

Contributions. The paper makes the following contributions.

- We give a reusable Agda representation of circuit presentations as word setoids: binary-tree words over generators, congruence closures of relations, and checked equational reasoning.
- We formalize generic normal-form interfaces and construction principles, including direct products, semidirect products, coset normal forms, amalgamation, and group extensions.
- We mechanize a normal-form theorem for $\text{Sp}(2n, \mathbb{Z}_p)$, parameterized by n and an odd prime p , including implementability and uniqueness of normal forms.
- We formalize the completeness proof pattern “push generators through boxes, then merge into normal form”, isolating the reusable normalization induction from the box-specific calculations.
- We derive Clifford-mod-scalar relations from Pauli and symplectic relations by semidirect product, prove a simplified presentation equivalent, and explain how to add scalar phases by a general group-extension recipe.
- We demonstrate the generality of the framework on amalgamation examples: single-qubit Clifford+ T , single-qutrit Clifford+ T , and the group of 3×3 unitary matrices over $\mathbb{Z}[1/2, i]$.

2 Circuit Presentations in Agda

The framework begins with a deliberately small syntactic representation. For a set X of generators, circuits are words:

$$w ::= [x] \mid \epsilon \mid w \cdot w.$$

The implementation uses binary trees rather than lists. This is not merely a programming convenience: in Agda, list concatenation forgets the exact operands used to build the list, while a binary tree preserves the local shape needed by congruence proofs. Associativity and unit laws are then added as relations, recovering the usual monoid behavior at the setoid level.

A presentation is a relation $\Gamma : \text{Word}(X) \rightarrow \text{Word}(X) \rightarrow \text{Set}$. The framework forms its congruence closure $w \approx_\Gamma v$, generated by reflexivity, symmetry, transitivity, congruence for composition, associativity, unit laws, and the axioms in Γ . This definition appears in `Presentation/Base.agda`. Standard Agda setoid reasoning is then used to write checked derivations that look like ordinary equational proofs.

2.1 Normal-Form Interfaces

The main abstraction is `NFProperty`. A presentation has an `NFProperty` when there is a type `NF`, a normalization function $\text{nf} : \text{Word}(X) \rightarrow \text{NF}$, a proof that equivalent words have equal normal forms, and a proof that equal normal forms imply word equivalence. The stronger `NFProperty'` additionally stores an inverse map from normal-form data back to words. These interfaces turn equality of circuits into equality of normal-form data, which is often decidable or structurally simpler.

TODO: Insert a small excerpt of `NFProperty` and `NFProperty'` from `Presentation/Properties.agda`, simplified for exposition.

2.2 Combinators for Presentations

The library contains construction principles that transport normal-form properties across larger presentations. Direct products combine independent presentations. Semidirect products add cross-relations expressing an action of one presentation on another. Coset normal forms extend a subgroup presentation by a finite or structured set of representatives. Amalgamation combines two presentations sharing a common subpresentation. Group extensions add a normal subgroup presentation, a quotient presentation, corrected lifts of quotient relations, and conjugation data.

These constructions are not merely organizational. They are the reason the Clifford and example developments do not need to reprove normal-form machinery from scratch. Each case supplies the local data required by the generic theorem, and Agda checks that the data respects the defining relations.

3 The Symplectic Case Study

Let p be an odd prime. The formalization represents a Pauli vector on n qudits as a vector of pairs in $\mathbb{Z}_p \times \mathbb{Z}_p$. For a single qudit, the symplectic form is

$$\langle (a, b), (c, d) \rangle = -ad + cb,$$

and the n -qudit form is the sum of these contributions across the vector. The module `N.Pauli` defines the Pauli vector type, the form, and basic lemmas such as antisymmetry and the behavior of the identity Pauli vector.

The generator set is indexed by arity. The single-qudit generators S and H live at every positive arity; the two-qudit generator CZ lives at arity at least two; and a lift operation shifts a generator to a higher wire. Derived words such as CX , $SWAP$, and multipliers are defined from these basic generators. The presentation `N.Symplectic` contains the parameterized relations used for the symplectic group.

3.1 Normal Boxes

The normal forms are built inductively from boxes. Informally, an n -qudit normal form consists of an $(n - 1)$ -qudit normal form lifted to the upper wires and an LM layer acting on the newly exposed wire. The boxes A, B, D, E, L, M encode structured fragments that eliminate or normalize the first nontrivial part of a symplectic tableau. In Agda these boxes are data types indexed by arity, with an interpretation as words.

TODO: Add a figure for the inductive normal form, adapted from Figure 3 of `doc/qudit-quantum-June12.pdf` if permissions/attribution are settled, or redraw as a simple schematic.

3.2 Implementability and Uniqueness

The semantic action of a word on Pauli vectors is defined generator by generator and extended compositionally. The formalization proves that this action is linear and preserves the symplectic form. Conversely, for every linear function preserving the symplectic form, `N.NF` constructs a normal form whose action is inverse to that function. Applying this construction to the inverse of a word yields a normal form implementing the word itself.

Uniqueness is proved by showing that distinct normal forms have distinct Pauli actions. The proof is inductive. At the outer layer, equality of actions implies equality of the head behavior of the LM component; a head-injectivity lemma identifies the LM boxes. Surjectivity of the tail action then reduces the problem to the inner normal forms. This is the core of `N.NF-Inj.agda`.

THEOREM 3.1 (NORMAL FORMS FOR $\text{Sp}(2n, \mathbb{Z}_p)$, MECHANIZED). *For every n and every odd prime p , every word over S , H , and CZ has a normal form with the same Pauli action. Moreover, two normal forms with the same action are equal.*

TODO: Replace this theorem statement with the exact names and dependencies once the trusted artifact slice is finalized.

4 Completeness by Pushing Through Boxes

The completeness proof separates box-specific algebra from a generic normalization induction. The box-specific part proves that each generator can be pushed through each normal box, producing a new box and some residual word. For example, pushing S through an E -box updates the parameter:

$$E_b \cdot S \approx E_{b-1}.$$

The theorem `BoxRelations.One.E<-S`, backed by `N.BR.One.E.lemma-single-qupit-br-E`, is the smallest example of a certified box reduction: it uses the single-qudit S -power lemma and arithmetic in $\mathbb{Z}_p$ to prove

$$[b]^e \cdot S \approx [b-1]^e.$$

At the other end of the scale, `BoxRelations.Three.DD<-CZ`, backed by `N.BR.Three.DD-CZ.lemma-dir-and-vd'`, pushes a CZ through a two-entry DD box:

$$[vd]^{vd} \cdot CZ \approx \text{dir}^\uparrow \cdot [vd']^{vd}.$$

The nonzero/nonzero cases require substantial reassociation, commutation, and movement of three-qudit gadgets. These examples illustrate why the proof assistant matters: the statement of the box relation is compact, but the checked derivation can be long and easy to miscopy by hand.

The generic part is implemented in `Symplectic-Completeness.agda`. The interface is a single operation:

$$\text{cosetUpdate}(lm, g) = (lm', w, [lm] \cdot [g] \approx w^\uparrow \cdot [lm']).$$

Given this operation, the rest of completeness is structural. To merge a gate into a normal form (nf, lm) , push the gate through lm , then recursively merge the residual word w into the inner normal form nf . To normalize a word, repeat this merge operation over the word structure.

THEOREM 4.1 (COMPLETENESS BY NORMALIZATION, MECHANIZED SKELETON). *Assuming the box-relation chaining operation `cosetUpdate`, every word appended to a normal form rewrites to a normal form. In particular, every word rewrites to a normal form by starting from the identity normal form.*

TODO: When `N.Completeness.lemma-coset-update` is finished, replace the assumption by the completed theorem and state the final completeness corollary: words modulo the symplectic relations are isomorphic to $\text{Sp}(2n, \mathbb{Z}_p)$.

5 From Symplectic to Clifford-Mod-Scalar

The project next derives Clifford-mod-scalar relations from two simpler presentations: a Pauli presentation and the symplectic presentation. The key structure is a semidirect product. The Pauli generators X and Z form the normal subgroup. The symplectic generators S , H , and CZ form the quotient layer. Cross-relations describe the conjugation action of symplectic generators on Pauli generators, for example how H swaps X and Z , how S maps X to XZ , and how CZ maps an X on one wire to an X together with a Z on the other.

This construction is formalized in `N.Clifford.SDProduct`. The combined generator set is the disjoint union of Pauli generators and symplectic generators. The combined relation is built using the generic semidirect-product presentation constructor. The embedding of the symplectic

presentation into the Clifford-mod-scalar presentation is the right injection into this disjoint union; the embedding of the Pauli presentation is the left injection. Thus a symplectic relation can be reused as a Clifford relation by mapping every symplectic generator through the embedding.

The comparison theorem `N.Clifford.Iso3.M.Theorem-SemiDirect-iso-Clifford` proves that the semidirect-product presentation is isomorphic to the Clifford-mod-scalar presentation. This theorem is the paper-level bridge between the generic algebraic construction and the concrete Clifford relations.

5.1 Simplifying the Clifford Presentation

The first Clifford-mod-scalar presentation uses derived words involving $s = SZ^{1/2}$ and multipliers. This is close to the semidirect-product derivation but not ideal as a final rule set. The simplified presentation pushes Pauli components to the right, uses basic S, H, CZ gates where possible, and adds explicit Z - CZ commutation rules to avoid circular derivations.

The file `N.Clifford.Clifford-Mod-Scalars-Simplified` states the simplified relations. The file `N.Clifford.Iso-Clifford-Simplified` proves that the original and simplified presentations are isomorphic by the identity map. The proof has two directions: every original relation is derivable in the simplified presentation, and every simplified relation is derivable in the original presentation.

A simple simplification example is `N.Clifford.Mg-Simplify.s-collect`: for every k , it collects the hidden Pauli part of s^k and rewrites it as

$$s^k \approx S^k \cdot Z^{k/2}.$$

This is the local shape of the entire simplification pass: expose the Pauli components, move them to a canonical side, and compare the remaining symplectic word. More complicated examples include the `final-semi-M*` theorems in `N.Clifford.Mg-Simplify`, which replace multiplier relations involving s by collected S, H -words with rightmost Pauli tails.

6 Adding Scalars as a General Group Extension

The semidirect-product construction above is split: quotient relations lift without correction. Adding scalar phases requires a more general group-extension presentation. Suppose N is a normal subgroup with presentation $\langle Y \mid S \rangle$, and the quotient G/N has presentation $\langle \bar{X} \mid \bar{R} \rangle$. Choosing representatives $X \subseteq G$ for the quotient generators gives two kinds of extra data. First, each quotient relator may lift to a word in N , giving a correction term. Second, each quotient generator conjugates each kernel generator to a word in N .

The module `Presentation.Group-Extension2` formalizes this recipe. The relations consist of kernel relations, corrected quotient relations, and conjugation relations. The split semidirect product is recovered as the special case where all quotient corrections are ϵ . Scalar phases are more general: quotient equations that hold only up to a scalar become corrected relations whose correction word lives in the scalar subgroup.

TODO: Instantiate this section with the scalar subgroup and the exact correction words for the Clifford presentation once those files are finalized.

7 Other Case Studies

The examples demonstrate that the library is not specialized to qudit Clifford circuits.

Each case builds smaller presentations with normal forms, constructs coset representatives or amalgamation data, and concludes with an isomorphism theorem between the intended presentation and the presentation generated by the generic construction.

The one-qubit Clifford+ T example ends with `Theorem-CliffordT1-iso-B*A**` in the final Examples. `CliffordT1.CliffordT1` module. It builds a direct product of cyclic presentations, extends it by coset

File	Object	Framework feature
Examples/CliffordT1.agda	one-qubit Clifford+ T	amalgamation
Examples/QuTritCliffordT1.agda	one-quTrit Clifford+ T	amalgamation
Examples/U33Di.agda	$U_3(\mathbb{Z}[1/2, i])$	cosets/amalgamation

Table 1. Independent case studies using the same presentation machinery.

normal forms, and then uses `AmalDataNF/ANF` to obtain the amalgamated product normal form and the final star-isomorphism theorem.

The one-quTrit Clifford+ T example has two final isomorphism theorems named `Theorem-CliffordT1-iso-B*A**`, one in `Examples.QuTritCliffordT1.CliffordT1` and one in the simplified module. Both are built from chained coset normal forms and the same amalgamation machinery.

The $U_3(\mathbb{Z}[1/2, i])$ example ends with `Examples.U33Di.Iso.Theorem-U33Di-iso-B*A**`. This case combines cyclic products, `SnD`, coset normal forms, direct products, sugar products, and amalgamation, making it the strongest evidence that the framework is not tied to Clifford-specific syntax.

8 Related Work

Circuit presentations and normal forms. Lafont’s algebraic theory of Boolean circuits is an early systematic study of generators and relations for circuit-like structures [21]. In the quantum setting, Selinger gave generators, relations, and normal forms for n -qubit Clifford operators [30]. Amy, Chen, and Ross gave a finite presentation for CNOT-dihedral operators using normal forms and rewrite systems [1]. Presentations have also been studied for larger exact-synthesis groups such as $U_4(\mathbb{Z}[1/\sqrt{2}, i])$ [16] and for two-qubit Clifford+ T operators [4]. These works are close in proof shape to ours, but their proofs are carried out mathematically rather than inside a proof assistant.

Qudit and Clifford completeness. The stabilizer and Clifford structures used here go back through work on higher-dimensional fault-tolerant quantum computation and arbitrary-dimensional Clifford operations [14, 18], as well as structural characterizations of Clifford operators [11] and normal-form decompositions of Clifford circuits [6]. Recent work gives complete and natural rule sets for multi-quTrit Clifford circuits [24]; the present formalization generalizes this style of normal-form reasoning to all odd prime dimensions at the symplectic level and mechanizes the algebra in Agda. The companion multi-qudit Clifford development for all odd primes uses the same normal boxes and relation-simplification strategy [3].

Beyond Clifford. Matsumoto and Amano introduced normal forms for single-qubit Clifford+ T circuits [25]; Giles and Selinger clarified this normal form and its exact-synthesis consequences [12]. Glaudell, Ross, and Taylor gave canonical forms for single-quTrit Clifford+ T operators [13], and Jain, Kalra, and Prakash extended normal-form ideas to single-qudit Clifford+ T operators [19]. Complete equational theories for broader circuit fragments include the general quantum-circuit theory of Clement et al. [8] and the real-Clifford+ CH theory of Clement [7]. These results motivate our future-work goal of formalizing the first n -qubit universal completeness theorem.

Graphical calculi. ZX-calculi provide complete graphical languages for several fragments of quantum mechanics, including stabilizer fragments and universal fragments [2, 9, 20]. The ZX tradition is complementary to our circuit-presentation approach: ZX systems reason in a richer graphical language, while this work focuses on syntactic presentations of circuit generators and

relations. Verified ZX efforts such as VyZX show a related proof-assistant direction for graphical languages.

Proof assistants for quantum circuits. QWIRE embeds quantum circuits in Coq as a core circuit language [26] and supports formal verification of circuit equivalences [28]; ReQWIRE extends this style of reasoning to reversible quantum circuits [27]. SQIR and VOQC develop a Coq-based intermediate representation and verified optimizer [17]; and VyZX formalizes ZX-calculus reasoning in Coq [22, 23]. Our work differs in its focus on presentations, normal forms, and completeness of circuit relations rather than semantic preservation of compiler optimizations.

Circuit languages and interoperability. Quipper, OpenQASM, and Qiskit provide practical circuit description and compilation ecosystems [10, 15, 29, 32]. Modern high-level languages such as Silq address programming abstractions like safe uncomputation [5]. Interfacing the Agda framework with such languages would allow external circuit programs to be imported into a certified presentation, normalized, and exported with proof-producing rewrites.

9 Future Work

First, the most direct extension is a formalization of normal forms and complete relations for Galois qudit Clifford circuits. Second, the current examples use at most binary gates; extending the framework to ternary and general n -ary gates would make it more suitable for universal circuit families and graphical calculi with arbitrary arities. Third, practical impact requires interfaces with circuit description languages such as Quipper, Qiskit, and OpenQASM. Finally, we plan to formalize the complete equational theory for real-Clifford+ CH circuits, the first completeness result for an n -qubit universal circuit fragment without parameterized gates or ancillas.

10 Artifact Scope

The repository contains both core developments and exploratory modules. For a POPL artifact, the trusted slice should be made explicit. The core slice should include the word/presentation framework, the symplectic normal-form modules, the box-relation derivations used by completeness, the Clifford-mod-scalar and simplified Clifford modules, the group-extension machinery, and the three example developments. The final artifact should either complete or exclude the Test modules in Examples/CliffordT1.agda and Examples/QuditCliffordT1.agda, which currently contain active holes; Examples/U33Di.agda appears clean. Exploratory files with holes, archived files, and older dimension-specific prototypes should be excluded from the artifact claim unless they are completed and added to the build.

References

- [1] Matthew Amy, Jianxin Chen, and Neil J. Ross. 2018. A Finite Presentation of CNOT-Dihedral Operators. In *Proceedings of the 14th International Conference on Quantum Physics and Logic (Electronic Proceedings in Theoretical Computer Science, Vol. 266)*. 84–97. doi:10.4204/EPTCS.266.6 arXiv:1701.00140 [quant-ph]
- [2] Miriam Backens. 2014. The ZX-Calculus is Complete for Stabilizer Quantum Mechanics. *New Journal of Physics* 16 (2014), 093021. doi:10.1088/1367-2630/16/9/093021
- [3] Xiaoning Bian, Sarah Meng Li, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2026. A Complete and Natural Rule Set for Multi-Qudit Clifford Circuits in All Odd Prime Dimensions. Manuscript.
- [4] Xiaoning Bian and Peter Selinger. 2022. Generators and Relations for 2-Qubit Clifford+T Operators. arXiv:2204.02217 [quant-ph]
- [5] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin T. Vechev. 2020. Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. 286–300. doi:10.1145/3385412.3386007
- [6] Sergey Bravyi and Dmitri Maslov. 2021. Hadamard-Free Circuits Expose the Structure of the Clifford Group. *IEEE Transactions on Information Theory* 67, 7 (2021), 4546–4563. doi:10.1109/TIT.2021.3081415 arXiv:2003.09412 [quant-ph]

- [7] Alexandre Clément. 2026. A Complete Equational Theory for Real-Clifford+CH Quantum Circuits. arXiv:2602.06644 [quant-ph]
- [8] Alexandre Clément, Nicolas Heurtel, Shane Mansfield, Simon Perdrix, and Benoît Valiron. 2022. A Complete Equational Theory for Quantum Circuits. arXiv:2206.10577 [quant-ph]
- [9] Bob Coecke and Ross Duncan. 2011. Interacting Quantum Observables: Categorical Algebra and Diagrammatics. *New Journal of Physics* 13 (2011), 043016. doi:10.1088/1367-2630/13/4/043016 arXiv:0906.4725 [quant-ph]
- [10] Andrew W. Cross, Ali Javadi-Abhari, Thomas Alexander, Niel de Beaudrap, Lev S. Bishop, Steven Heidel, Colm A. Ryan, Prasahnt Sivarajah, John Smolin, Jay M. Gambetta, and Blake R. Johnson. 2022. OpenQASM 3: A Broader and Deeper Quantum Assembly Language. *ACM Transactions on Quantum Computing* 3, 3, Article 12 (2022), 50 pages. doi:10.1145/3505636 arXiv:2104.14722 [quant-ph]
- [11] J. M. Farinholt. 2014. An Ideal Characterization of the Clifford Operators. *Journal of Physics A: Mathematical and Theoretical* 47, 30 (2014), 305303. doi:10.1088/1751-8113/47/30/305303 arXiv:1307.5087 [quant-ph]
- [12] Brett Giles and Peter Selinger. 2013. Remarks on Matsumoto and Amano’s Normal Form for Single-Qubit Clifford+T Operators. arXiv:1312.6584 [quant-ph]
- [13] Andrew N. Glaudell, Neil J. Ross, and Jacob M. Taylor. 2019. Canonical Forms for Single-Qutrit Clifford+T Operators. *Annals of Physics* 406 (2019), 54–70. doi:10.1016/j.aop.2019.04.009 arXiv:1803.05047 [quant-ph]
- [14] Daniel Gottesman. 1999. Fault-Tolerant Quantum Computation with Higher-Dimensional Systems. *Chaos, Solitons & Fractals* 10, 10 (1999), 1749–1758. doi:10.1016/S0960-0779(98)00218-5 arXiv:quant-ph/9802007
- [15] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. 2013. Quipper: A Scalable Quantum Programming Language. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 333–342. doi:10.1145/2491956.2462177 arXiv:1304.3390 [cs.PL]
- [16] Seth Eveson Murray Greylyn. 2014. Generators and Relations for the Group $U_4(\mathbb{Z}[1/\sqrt{2}, i])$. arXiv:1408.6204 [quant-ph]
- [17] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. 2021. A Verified Optimizer for Quantum Circuits. *Proceedings of the ACM on Programming Languages* 5, POPL, Article 37 (2021), 29 pages. doi:10.1145/3434318 arXiv:1912.02250 [cs.PL]
- [18] Erik Hostens, Jeroen Dehaene, and Bart De Moor. 2005. Stabilizer States and Clifford Operations for Systems of Arbitrary Dimensions, and Modular Arithmetic. *Physical Review A* 71 (2005), 042315. doi:10.1103/PhysRevA.71.042315 arXiv:quant-ph/0408190
- [19] Akalank Jain, Amol Ratan Kalra, and Shiroman Prakash. 2020. A Normal Form for Single-Qudit Clifford+T Operators. arXiv:2011.07970 [quant-ph]
- [20] Emmanuel Jeandel, Simon Perdrix, and Renaud Vilmart. 2018. A Complete Axiomatisation of the ZX-Calculus for Clifford+T Quantum Mechanics. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*. 559–568. doi:10.1145/3209108.3209131
- [21] Yves Lafont. 2003. Towards an Algebraic Theory of Boolean Circuits. *Journal of Pure and Applied Algebra* 184, 2–3 (2003), 257–310.
- [22] Adrian Lehmann, Ben Caldwell, and Robert Rand. 2022. VyZX: A Vision for Verifying the ZX Calculus. arXiv:2205.05781 [cs.LO]
- [23] Adrian Lehmann, Ben Caldwell, Bhakti Shah, and Robert Rand. 2023. VyZX: Formal Verification of a Graphical Quantum Language. arXiv:2311.11571 [cs.LO]
- [24] Sarah Meng Li, Michele Mosca, Neil J. Ross, John van de Wetering, and Yuming Zhao. 2025. A Complete and Natural Rule Set for Multi-Qutrit Clifford Circuits. arXiv:2508.14670 [quant-ph]
- [25] Ken Matsumoto and Kazuyuki Amano. 2008. Representation of Quantum Circuits with Clifford and $\pi/8$ Gates. arXiv:0806.3834 [quant-ph]
- [26] Jennifer Paykin, Robert Rand, and Steve Zdancewic. 2017. QWIRE: A Core Language for Quantum Circuits. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. 846–858. doi:10.1145/3009837.3009894
- [27] Robert Rand, Jennifer Paykin, Dong-Ho Lee, and Steve Zdancewic. 2019. ReQWIRE: Reasoning about Reversible Quantum Circuits. arXiv:1901.10118 [cs.PL]
- [28] Robert Rand, Jennifer Paykin, and Steve Zdancewic. 2018. QWIRE Practice: Formal Verification of Quantum Circuits in Coq. In *Proceedings of the 14th International Conference on Quantum Physics and Logic (Electronic Proceedings in Theoretical Computer Science, Vol. 266)*. 119–132. doi:10.4204/EPTCS.266.8 arXiv:1803.00699 [cs.PL]
- [29] Peter Selinger. 2004. Towards a Quantum Programming Language. *Mathematical Structures in Computer Science* 14, 4 (2004), 527–586. doi:10.1017/S0960129504004256
- [30] Peter Selinger. 2015. Generators and Relations for n -Qubit Clifford Operators. *Logical Methods in Computer Science* 11, 2 (2015), 10:1–10:17. doi:10.2168/LMCS-11(2:10)2015 arXiv:1310.6813 [quant-ph]
- [31] The Agda Team. 2026. Agda Documentation. <https://agda.readthedocs.io/>. Accessed 2026-06-28.

[32] Robert Wille, Rod Van Meter, and Yehuda Naveh. 2019. IBM’s Qiskit Tool Chain: Working with and Developing for Real Quantum Computers. In *Design, Automation & Test in Europe Conference & Exhibition*. 1234–1240. doi:10.23919/DATE.2019.8715261